

COMMITERS

COMMIT. CODE. CONNECT.

Idea to Action Plan: QR Review Booster

Solo Developer Blueprint & Accelerated Technical Strategy

This document serves as the official, executable technical action plan for launching the ****Commiters Local Review Booster****. Designed specifically for solo execution, it prioritizes speed-to-market, zero communication overhead, and architectural efficiency to capture local business footprints in Udaipur.

1. The Idea (Objective)

The objective is to deploy a high-utility, lightning-fast ****Micro-SaaS Responsive Web App**** that helps local hospitality, retail, and service businesses skyrocket their positive Google reviews while privately handling customer complaints.

By eliminating the download barrier of mobile apps, we establish a zero-friction channel where customers scan physical table QR codes and rate their experience in under 10 seconds.

Core Value Metric

- **For Local Businesses:** Direct organic search boost on Google Maps (ranking high in local search queries) and real-time alerts to resolve physical service issues table-side.
- **For Commiters:** Seamless, self-funded B2B lead generation. A subtle "Powered by Commiters" link in the footer puts the agency directly in front of visiting tech founders, business owners, and local entrepreneurs.

2. The Solo Developer Roadmap (7-Day Sprint)

To maximize solo momentum and ensure rapid time-to-market, development is structured into distinct, daily modular tasks:

1. Day 1: Database Setup & Core API Route (Backend Core)

- Initialize the database schema (PostgreSQL via Supabase or Neon).
- Write the base CRUD operations utilizing Prisma Client.
- Write API endpoints for basic merchant signup and feedback submission.

2. Day 2: Customer Rating Page & Compliance Flow (Frontend Core)

- Build a clean, lightning-fast, mobile-first star rating page (<200ms target load).
- Implement the conditional branching routing: 4-5 stars forward to Google Maps; 1-3 stars load the local feedback form.
- Include the compliant, low-friction option for negative raters to access Google Reviews directly, keeping the system 100% compliant.

3. Day 3: Merchant Dashboard & QR Generator

- Create the admin interface for merchants to sign in, view logged feedback, and download custom dynamic QR codes tied to unique locations (e.g., "table-1").

4. Day 4: Real-Time WhatsApp Integration

- Hook into a WhatsApp business API gateway (Twilio or Gupshup).
- Implement a serverless notification trigger: if a rating is ≤ 3 , immediately dispatch the comments, table ID, and customer info to the owner's WhatsApp.

5. Day 5: Security Hardening & Edge Deployments

- Deploy to Vercel production edge runtimes.
- Implement basic API rate-limiting on feedback submissions to prevent spam attacks on merchants' Google profiles.

6. Day 6 & 7: Beta Trials & Local Launch

- Deploy to a staging domain and print 5-10 sample acrylic T-stands locally in Udaipur.
- Introduce the tool to a friendly local lakeside cafe for a 30-day trial to secure initial production telemetry.

3. System Architecture

The product leverages a serverless, state-of-the-art tech stack matching the high engineering standards of the Committers brand.

High-Level System Design

- **Frontend / Framework:** Next.js (App Router) for Server-Side Rendering (SSR) to render custom customer pages on the edge instantly.
- **Database / ORM:** Prisma ORM interacting with a PostgreSQL relational database on Supabase for lightweight storage.
- **Styling:** Tailwind CSS to keep code modular and layout fully responsive.
- **Async Communication:** Twilio/Gupshup serverless webhook integrations.

Prisma Data Model

```
model Business {
  id          String    @id @default(uuid())
  name        String
  email       String    @unique
  passwordHash String
  whatsappNum String?
  googleReviewUrl String
  createdAt   DateTime  @default(now())

  qrCodes     QRCode[]
  feedbacks    Feedback[]
}

model QRCode {
  id          String    @id @default(uuid())
  identifier   String    @unique // e.g., "table-5"
  location     String?
  businessId   String

  business     Business @relation(fields: [businessId], references: [id], onDelete: Cascade)
  feedbacks    Feedback[]
}

model Feedback {
  id          String    @id @default(uuid())
  stars       Int        // 1 to 5
  customerName String?
  customerPhone String?
  comments    String?
  isPublicRedirect Boolean @default(false)
  createdAt   DateTime  @default(now())

  businessId   String
  business     Business @relation(fields: [businessId], references: [id], onDelete: Cascade)
  qrCodeId     String?
  qrCode       QRCode?  @relation(fields: [qrCodeId], references: [id], onDelete: SetNull)
}
```